From Probability to Assistant: How Language Models Work and How They Learn to Help

There is a single idea at the heart of every modern language model, and it is almost disappointingly simple: given some text, guess the next token. ChatGPT, Claude, and Grok are all, at their core, machines that make this one guess over and over. Everything else — the mathematics, the architecture, the training regimes — is machinery in service of making that guess well. This essay traces the full arc of that machinery in four movements: first, the model as a probability engine; second, how text becomes math; third, how math becomes text again; and finally, how a raw predictor is transformed into the assistant you actually talk to.

I. The Model Is a Probability Engine

Language models are *autoregressive*: they predict the next token from everything that came before, append that prediction to the sequence, and repeat. Formally, at each step the model computes

$$P(x_t \mid x_1, x_2, \ldots, x_{t-1})$$

— the probability of the next token x_t given the entire preceding sequence. Type a prompt, and the model generates its reply one token at a time, each new token conditioned on the growing text behind it. The loop is humble, but it is the operating principle of every major chatbot in use today.

The architectural breakthrough that made this loop powerful is *attention*, introduced in the 2017 paper "Attention Is All You Need." For each prediction, the model does not treat all prior words equally; it learns to weight which earlier words matter most for the guess at hand. When completing "The trophy didn't fit in the suitcase because *it* was too big," attention is what lets the model look back and decide which noun "it" refers to. This mechanism — dynamically deciding what to attend to — is the foundation of the modern large language model.

But to understand what the model is actually doing when it "guesses," we need the language of probability. A *probability distribution* is a complete list of possible outcomes, each assigned a likelihood, with the likelihoods summing to one. A fair six-sided die assigns $P(i) = \frac{1}{6}$ to each face, and in general any distribution must satisfy

$$\sum_i P(x_i) = 1, \qquad P(x_i) \geq 0.$$

A language model, at every step, produces exactly this kind of object: a distribution over every token in its vocabulary.

Sampling is the act of drawing one outcome from that distribution — rolling the die. This is why the same prompt can yield different answers on different runs: the model is not retrieving a stored response, it is sampling from a distribution, and randomness is built into the draw.

Where does the distribution come from? This is the leap from classical probability to *generative modeling*. Rather than hand-writing the probabilities, as we can for a die, we *estimate* them from data. Training a generative model means learning a distribution from examples. And because language is *sequential data*, the relevant question is always conditional: what is the probability of the next event, given the sequence so far?

Here the mathematics delivers an elegant guarantee. The *chain rule of probability* tells us that the joint probability of an entire sequence factors into a product of conditional probabilities:

$$P(x_1, x_2, \ldots, x_n) = P(x_1) \cdot P(x_2 \mid x_1) \cdot P(x_3 \mid x_1, x_2) \cdots P(x_n \mid x_1, \ldots, x_{n-1}) = \prod_{t=1}^{n} P(x_t \mid x_{<t}).$$

This means that sampling a sequence token-by-token, autoregressively, is mathematically equivalent to sampling the entire sequence from the joint distribution at once. The humble one-token-at-a-time loop is not a hack; it is a faithful implementation of sampling from a distribution over whole texts. The two-variable case is the familiar identity

$$P(x, y) = P(x \mid y) \cdot P(y),$$

from which Bayes' rule follows by symmetry:

$$P(y \mid x) = \frac{P(x \mid y)\, P(y)}{P(x)}.$$

The chain rule above is simply this machinery extended across n tokens.

A consequence follows immediately: models mirror their training data. A model trained on the collected works of Jane Austen will write like Jane Austen, because the distribution it has learned *is* the distribution of Austen's prose. Scale this up to the internet, and the model learns the distribution of human text at large — with all its knowledge, styles, and biases.

Something surprising happens at that scale. Optimizing nothing but next-token prediction, models develop abilities nobody explicitly taught them — translation, arithmetic, rudimentary reasoning. This is *emergence*: capabilities that arise as a side effect of learning the distribution of human text well enough. The flip side is *overfitting*: with too little data, a model memorizes its training examples rather than generalizing from them, reciting rather than understanding.

If the model is a completion machine, how do you steer it? At inference time there is exactly one steering wheel: *the prompt*. You control a completion machine by writing the beginning of the text and letting the model continue it. Prompt engineering is nothing more mysterious than choosing a beginning whose most probable continuations are the ones you want.

II. How Text Becomes Math

Neural networks compute with numbers, not words, so the first task is translation. *Tokenization* chops text into chunks — typically via byte-pair encoding, which builds a vocabulary of common character sequences — and maps each chunk to an ID in a fixed dictionary. Tokens are usually word fragments rather than whole words. This detail explains two everyday quirks: why models have token limits (the context window is measured in tokens, not words), and why they can stumble on tasks like counting the letters in "strawberry" (the model sees token IDs, not individual characters).

A token ID alone carries no meaning. The crudest way to represent it numerically is *one-hot encoding*: a vector of zeros with a single 1 in the position of the token's ID. For a vocabulary of size V, the token with ID k becomes

$$\mathbf{e}_k = [\,0,\ 0,\ \ldots,\ \underbrace{1}_{\text{position } k},\ \ldots,\ 0\,] \in \mathbb{R}^V.$$

One-hot vectors work, but they are enormous, and worse, they encode no relationships: every pair of distinct one-hot vectors is orthogonal ($\mathbf{e}_i \cdot \mathbf{e}_j = 0$ for $i \neq j$), so "cat" and "kitten" are exactly as distant as "cat" and "carburetor." One-hot encoding is best understood as the problem that embeddings solve.

Embeddings are the solution: each token is mapped to a point in a continuous space of roughly one to three thousand dimensions, $\mathbf{v} \in \mathbb{R}^d$ with $d \approx 1000$–3000, and the positions are *learned* so that related words end up near each other. Distance becomes a proxy for relatedness, and directions in the space capture relationships. The classic demonstration is that the displacement from "king" to "queen" roughly parallels the displacement from "man" to "woman":

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}},$$

or equivalently $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{queen}} \approx \mathbf{v}_{\text{man}} - \mathbf{v}_{\text{woman}}$: a single direction in the space encodes gender, independent of royalty. This is the moment meaning becomes geometry.

One ingredient is still missing: order. "The cat chased the dog" and "the dog chased the cat" contain identical tokens but mean opposite things. *Positional embeddings* inject word order into the representation, either by encoding each token's absolute position or its position relative to other tokens.

With tokens embedded and positioned, the *transformer architecture* takes over: a stack of repeating blocks, each combining an attention layer (which lets tokens exchange information based on learned relevance) with a feedforward layer (which processes each position's representation). The attention operation itself has a compact closed form — each token issues a *query* Q, is indexed by a *key* K, and carries a *value* V, and

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where the softmax converts query–key similarity scores into weights that decide how much each earlier token's value contributes. Stack these blocks dozens of times, and you have the full architecture of a modern language model.

III. How Math Becomes Text Again

The transformer's final output, for each possible next token, is a raw score called a *logit*. Logits are not yet probabilities — they can be negative, and they do not sum to one. They are the model's unnormalized opinion about every candidate token.

The *softmax* function converts logits into a proper probability distribution: it exponentiates each score and normalizes so the results sum to one. Given logits $z_1, \ldots, z_V$ over the vocabulary,

$$P(x_i) = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}.$$

Now the model's opinion is a genuine distribution we can sample from — we are back in the world of Part I.

Before sampling, we can adjust a dial: *temperature*. Dividing the logits by a temperature T before the softmax reshapes the distribution:

$$P(x_i) = \frac{e^{z_i/T}}{\sum_{j=1}^{V} e^{z_j/T}}.$$

As $T \to 0$ the distribution collapses toward the single highest-logit token (greedy behavior); as T grows it flattens toward uniform. Low temperature sharpens it, concentrating probability on the top candidates and making output predictable and conservative. High temperature flattens it, spreading probability across more candidates and making output more creative — and more error-prone. Temperature is not a metaphor; it is a literal parameter you set in an API call.

Finally, the decoding decision: *greedy decoding* always takes the single highest-probability token, $x_t = \arg\max_i P(x_i)$, while *sampling* draws $x_t \sim P$ from the distribution. Greedy decoding is deterministic but can be dull and repetitive; sampling introduces variety at the cost of occasional bad draws. The "NPRD" glitch from lecture illustrates both the risk and the recovery: a low-probability sample can produce a garbled token, but because every subsequent prediction is conditioned on the full context — chain rule again — the model can absorb the mistake and steer the continuation back to coherence.

Notice the shape of the full loop. Text is tokenized, embedded, and positioned (movement II); the transformer processes it and emits logits, which softmax turns into a distribution, from which we sample a token (movement III); the token is appended and the loop repeats (movement I). Every sentence a language model has ever produced is this loop, run again and again.

IV. From Raw Predictor to Assistant

A model trained only to predict the next token of internet text is not an assistant. Ask it a question and it may continue with *more questions*, because on the internet, questions often appear in lists of questions. Part 2 of the course is about closing the gap between "excellent text continuer" and "helpful assistant."

First, the loss function that drives all training. *Entropy* measures the uncertainty in a distribution:

$$H(p) = -\sum_i p(x_i) \log p(x_i).$$

A distribution concentrated on few outcomes has low entropy; one spread across many has high entropy — the sum of two dice piles probability near 7, giving it lower entropy than a single uniform die. *Cross-entropy* compares a predicted distribution q against the true one p:

$$H(p, q) = -\sum_i p(x_i) \log q(x_i).$$

This is not merely related to training — it *is* the training loss. In next-token prediction, the "true" distribution places all its mass on the token that actually appears, so the loss for one step reduces

to $-\log q(x_t^*)$: the negative log-probability the model assigned to the correct token. Every gradient step in pre-training is a step toward lower cross-entropy between the model's predictions and the data.

The modern *training pipeline* has three stages. *Pre-training* is the internet-scale next-token prediction we have discussed, and it is where essentially all of the model's knowledge and capability comes from. *Supervised fine-tuning* (SFT) then trains the model on human-written demonstrations of good assistant behavior — questions paired with helpful answers — teaching it the *format* of being an assistant. *Reinforcement learning from human feedback* (RLHF) refines this further: humans rank pairs of model outputs, a reward model $r(x, y)$ is trained to predict those rankings, and the language model π_θ is optimized to maximize the learned reward — typically with a penalty term that keeps it from drifting too far from the SFT model:

$$\max_\theta \; \mathbb{E}_{y\sim\pi_\theta}\big[\, r(x,y)\,\big] - \beta\, D_{\mathrm{KL}}\big(\pi_\theta \,\|\, \pi_{\mathrm{SFT}}\big).$$

It is worth pausing on the human side of this pipeline: the rankings and demonstrations come from large workforces of data labelers, and the labor conditions of that work — who does it, what they are paid, what content they are exposed to — is a live ethical issue in the field.

This pipeline gives us the distinction between *base* and *aligned* models. A base model is the raw text continuer: no assistant persona, no refusals, no safety behavior. An aligned model is the base model with SFT and RLHF layered on top. The crucial asymmetry is that *capability lives in the base model, while safety behavior is a layer added afterward.* This is precisely why jailbreaks work: a clever prompt does not teach the model anything new — it finds a path around the alignment layer to capabilities the base model always had.

The *superficial alignment hypothesis* pushes this idea to its logical end: perhaps all of a model's knowledge and ability comes from pre-training, and alignment does nothing but teach the model which capabilities to surface and in what style. If true, alignment is a thin veneer over a fixed underlying system — a possibility with real governance implications, since it suggests safety properties may be easier to strip away than to build in. Whether the hypothesis holds in full is an open research question.

Finally, the field keeps simplifying its own machinery. *Direct Preference Optimization* (DPO) achieves RLHF-like results without training a separate reward model at all, working directly from preference pairs. The insight behind it is captured in the title of the DPO paper's central claim: your language model is secretly a reward model already. DPO's simplicity has made it widely adopted, especially in the open-source community.

One major topic — LLM *reasoning*, including chain-of-thought prompting and dedicated reasoning models — was cut for time. It is currently the most active research area in the field, and Kevin's recording will cover it when posted.

Four Movements, One Machine

Step back and the whole course so far resolves into a single picture. A language model is a probability engine (I–II of Part 1): it learns a distribution over text and generates by sampling from it, one token at a time, with the chain rule guaranteeing that this is a principled thing to do. To run that engine, text must become math — tokens, embeddings, positions, transformer blocks — and math must become text again — logits, softmax, temperature, a sampling decision. And to turn the

resulting raw predictor into something you would actually want to talk to, we layer alignment on top: fine-tuning on demonstrations, optimizing against human preferences, and confronting honest open questions about how deep that layer really goes.

Every conversation you have with a chatbot is this entire stack, executing in full, once per token. The magic is not in any single component. It is in the fact that "guess the next token, well enough, at scale" turns out to contain so much.